How High Level Abstraction Was Implemented Using Inheritance — Deep Dive

The transcript showed a specific step by step thought process. Let me trace through every single step exactly as it was built.

The Starting Point — The Problem

You have two completely different types of cars:

```
class FuelCar {
    void drive() {
        // Combustion engine logic
        System.out.println("Fuel car driving - combustion");
    }
}

class ElectricCar {
    void drive() {
        // Electric motor logic
        System.out.println("Electric car driving - electric motor");
    }
}
```

Both have `drive()`. Both do it differently.

Now in your main code:

```
FuelCar fc = new FuelCar();
ElectricCar ec = new ElectricCar();

fc.drive(); // Only ever calls FuelCar's drive
ec.drive(); // Only ever calls ElectricCar's drive
```

What is the problem here?

`fc` is PERMANENTLY, RIGIDLY tied to `FuelCar`. `ec` is PERMANENTLY, RIGIDLY tied to `ElectricCar`.

You need TWO separate variables. TWO separate types. If tomorrow you add `DieselCar`, `HybridCar`, `HydrogenCar` — you need five separate variables, five separate types.

The WHAT (`drive a car`) and the HOW (`combustion` vs `electric`) are **tightly coupled.**

```
fc.drive()  ──permanently──>  FuelCar's drive code
ec.drive()  ──permanently──>  ElectricCar's drive code
```

Can never swap. Can never share one variable. Complete rigidity.

The Goal — What We Want to Achieve

The transcript said we want ONE reference variable `c` of type `Car` that can:

- Point to a FuelCar object sometimes
- Point to an ElectricCar object sometimes
- Call `drive()` and automatically use the RIGHT implementation

```
Car c = new FuelCar();
c.drive(); // Uses FuelCar's drive

c = new ElectricCar(); // Just change this ONE thing
c.drive(); // Now uses ElectricCar's drive
```

Same variable. Same method call. Different behavior. WHAT separated from HOW.

How do we achieve this using inheritance?

Step 1 — Create a Parent Class Called Car

The first thing done was creating a common parent:

```
class Car {
   void drive() {
      System.out.println("Generic car driving"); // some code here
```

```
    }
}
```

Now make both classes extend it:

```
class FuelCar extends Car {
    @Override
    void drive() {
        System.out.println("Fuel car - combustion engine");
    }
}

class ElectricCar extends Car {
    @Override
    void drive() {
        System.out.println("Electric car - electric motor");
    }
}
```

Now the hierarchy looks like:

```
      Car
      /   \
  FuelCar  ElectricCar
```

Step 2 — Use Parent Reference to Point to Child Objects

This is where inheritance enables the magic.

Because `FuelCar extends Car`, Java knows: **FuelCar IS-A Car**

Because `ElectricCar extends Car`, Java knows: **ElectricCar IS-A Car**

So wherever Java expects a `Car`, you can give either a `FuelCar` or `ElectricCar`.

```
Car c = new FuelCar();    // Valid — FuelCar IS-A Car
Car c = new ElectricCar(); // Valid — ElectricCar IS-A Car
```

This is **upcasting** — treating child object as parent type.

Step 3 — Call drive() Through Parent Reference

```
Car c = new FuelCar();
c.drive();
```

Now what happens when `c.drive()` is called?

Java looks at the **actual object in heap memory** — which is a `FuelCar`. `FuelCar` has its own `drive()`. So `FuelCar`'s `drive()` runs.

```
Car c = new ElectricCar();
c.drive();
```

Now the actual object is `ElectricCar`. So `ElectricCar`'s `drive()` runs.

Same call `c.drive()`. Different actual behavior. **WHAT separated from HOW.**

Step 4 — What About Common Methods?

The transcript made an important point here. Not everything is different between FuelCar and ElectricCar.

For example — applying the handbrake. Both cars do it the same way. Lock the rear wheels. Same mechanism.

```
class Car {
    // Common method — works same for ALL cars
    void applyHandBrake() {
        System.out.println("Handbrake applied - rear wheels locked");
    }

    // Different for each car — just declare existence here
    void drive() {
        System.out.println("Generic driving"); // problem — meaningless code
    }
}

class FuelCar extends Car {
```

```
    @Override
    void drive() {
        System.out.println("Combustion engine driving");
    }
    // applyHandBrake() inherited automatically — no need to rewrite
}

class ElectricCar extends Car {
    @Override
    void drive() {
        System.out.println("Electric motor driving");
    }
    // applyHandBrake() inherited automatically — no need to rewrite
}
```

Now:

```
Car c = new FuelCar();
c.drive();          // FuelCar's drive — specific HOW
c.applyHandBrake();  // Car's handbrake — common for all

c = new ElectricCar();
c.drive();          // ElectricCar's drive — different HOW
c.applyHandBrake();  // Car's handbrake — same common method
```

Inheritance gave us:

- **Common behavior** (`applyHandBrake`) — defined ONCE in parent, inherited by all children
- **Different behavior** (`drive`) — each child provides its own version

---

Step 5 — The Full Picture of How It Achieves Separation

Let's draw exactly what happens in memory:

```
Car c = new FuelCar();
c.drive();

STACK            HEAP
_______          ____________________________________
```

```
c ———>        [ FuelCar Object        ]
(Car type)    [ drive() ← FuelCar's    ]
              [ applyHandBrake() ← Car ]
```

When `c.drive()` is called:

1. Java looks at what `c` actually points to in heap
2. Finds a `FuelCar` object
3. Checks — does `FuelCar` have `drive()`? YES
4. Calls `FuelCar`'s `drive()`

```
c = new ElectricCar(); // Change the object
c.drive();
```

```
STACK          HEAP
———————        ————————————————————————————

c ————————>    [ ElectricCar Object     ]
(Car type)     [ drive() ← ElectricCar's]
               [ applyHandBrake() ← Car ]
```

When `c.drive()` is called now:

1. Java looks at what `c` actually points to
2. Finds an `ElectricCar` object
3. Checks — does `ElectricCar` have `drive()`? YES
4. Calls `ElectricCar`'s `drive()`

The reference `c` never changed its type. Still `Car`. But the object it points to changed. And the method that runs changes accordingly.

This is how WHAT got separated from HOW.

- WHAT = `c.drive()` — this line never changes
- HOW = depends on what object `c` points to — changes dynamically

---

Step 6 — Switching at Runtime

The most powerful demonstration. The object can be decided based on runtime conditions:

```
String userInput = scanner.nextLine(); // User types "fuel" or "electric"

Car c; // Just a reference, no object yet

if (userInput.equals("fuel")) {
    c = new FuelCar();     // HOW decided at runtime
} else {
    c = new ElectricCar(); // HOW decided at runtime
}

// This line NEVER changes regardless of which car was chosen:
c.drive(); // WHAT stays the same
```

The WHAT (`c.drive()`) is written once. The HOW (which specific `drive()` runs) is decided while the program is running based on real world input. Complete separation achieved.

But Wait — The Problem That Still Remains

The transcript then pointed out that using plain inheritance still has one big problem.

Look at the parent `Car` class:

```
class Car {
    void drive() {
        System.out.println("Generic car driving"); // WHAT IS THIS??
    }
}
```

This code is meaningless and problematic:

Problem 1: What do you write here? Car is just a concept. There is no "generic driving." Every real car drives differently.

Problem 2: Someone can accidentally create a `Car` object directly:

```
Car c = new Car(); // Should not be allowed — Car is just a concept
c.drive();         // Runs meaningless generic code — wrong behavior
```

Problem 3: No one is FORCED to override `drive()`. A developer can create:

```
class DieselCar extends Car {
    // Forgot to override drive()
    // No error from Java
    // Silently uses parent's meaningless generic code
}

Car c = new DieselCar();
c.drive(); // Runs Car's generic code — wrong, but no error
```

These are exactly the problems that abstract classes solve.

Plain inheritance gave us the MECHANISM (parent reference pointing to child objects) but didn't give us the SAFETY and ENFORCEMENT.

The Upgrade — From Plain Inheritance to Abstract Class

The transcript then showed the upgrade:

```
// BEFORE — plain inheritance (problems exist)
class Car {
    void drive() {
        System.out.println("meaningless generic code"); // bad
    }
}

// AFTER — abstract class (problems solved)
abstract class Car {
    abstract void drive(); // No meaningless code. Just declaration.

    void applyHandBrake() { // Common method stays defined
        System.out.println("Handbrake applied");
    }
}
```

Now:

```
Car c = new Car();      // COMPILE ERROR — cannot instantiate abstract class
Car c = new FuelCar();  // Works — FuelCar is concrete
```

And if someone creates `DieselCar` without overriding `drive()`:

```
class DieselCar extends Car {
    // Forgot drive() — COMPILE ERROR immediately
    // "DieselCar must implement inherited abstract method Car.drive()"
}
```

The compiler forces correct behavior.

The Complete Final Code

```
// Parent — abstract class
// Defines WHAT (declares drive)
// Implements common HOW (applyHandBrake)
abstract class Car {

    // WHAT — declared, no HOW here
    abstract void drive();
    abstract void brake();

    // Common HOW — same for all cars
    void applyHandBrake() {
        System.out.println("Handbrake applied — rear wheels locked");
    }

    void start() {
        System.out.println("Car starting...");
    }
}

// Child 1 — concrete class
// Provides its specific HOW
class FuelCar extends Car {

    @Override
    void drive() {
        System.out.println("FuelCar: combustion engine firing, petrol burning");
    }

    @Override
    void brake() {
```

```
        System.out.println("FuelCar: hydraulic disc brakes engaged");
    }
}

// Child 2 — concrete class
// Provides its different HOW
class ElectricCar extends Car {

    @Override
    void drive() {
        System.out.println("ElectricCar: electric motor spinning, battery draining");
    }

    @Override
    void brake() {
        System.out.println("ElectricCar: regenerative braking, battery charging!");
    }
}

// Child 3 — concrete class
class DieselCar extends Car {

    @Override
    void drive() {
        System.out.println("DieselCar: diesel combustion, torque power");
    }

    @Override
    void brake() {
        System.out.println("DieselCar: air brakes engaged");
    }
}

// Main — demonstrates separation of WHAT from HOW
public class Main {

    // This method demonstrates the power completely
    // It accepts ANY car — doesn't care which specific type
    static void testDrive(Car c) {
        c.start();          // Common — always same
        c.drive();          // WHAT is same, HOW depends on actual object
        c.brake();          // WHAT is same, HOW depends on actual object
        c.applyHandBrake(); // Common — always same
        System.out.println("---");
```

```
    }

    public static void main(String[] args) {

        // WHAT (testDrive call) stays same
        // HOW (actual car type) changes
        testDrive(new FuelCar());    // FuelCar's HOW
        testDrive(new ElectricCar()); // ElectricCar's HOW
        testDrive(new DieselCar());   // DieselCar's HOW

        // One reference, multiple HOWs
        Car c;

        c = new FuelCar();
        c.drive(); // Combustion

        c = new ElectricCar(); // Same variable c, just reassigned
        c.drive(); // Electric motor

        c = new DieselCar(); // Same variable c again
        c.drive(); // Diesel

        // Car c = new Car(); // COMPILE ERROR — abstract, good!
    }
}
```

Output:

```
Car starting...
FuelCar: combustion engine firing, petrol burning
FuelCar: hydraulic disc brakes engaged
Handbrake applied — rear wheels locked
---
Car starting...
ElectricCar: electric motor spinning, battery draining
ElectricCar: regenerative braking, battery charging!
Handbrake applied — rear wheels locked
---
Car starting...
DieselCar: diesel combustion, torque power
DieselCar: air brakes engaged
Handbrake applied — rear wheels locked
---
```

The Exact Separation That Was Achieved

The Exact Separation That Was Achieved

```
WHAT (never changes)          HOW (changes dynamically)
____________________          _________________________

c.drive()          ------>    FuelCar's drive()
                   OR ------> ElectricCar's drive()
                   OR ------> DieselCar's drive()

c.brake()          ------>    FuelCar's brake()
                   OR ------> ElectricCar's brake()
                   OR ------> DieselCar's brake()

c.applyHandBrake() ------>    Always Car's applyHandBrake()
                              (common, never changes)
```

The WHAT lines are fixed in your code. The HOW lines are decided at runtime based on what object `c` actually points to. That is the complete separation. That is high level abstraction implemented through inheritance.